MoodMix: Mood-Based Playlist Generator

DBMS Experiential Learning - Level 3 Demonstration Report

Team: Nitin Somisetty (PES2UG24CS327), Nishit Suresh (PES2UG24CS326)

Project repository: NitinSomisetty/Mood-Based-Playlist-Generator-

Technology: Next.js 14, TypeScript, PostgreSQL, Supabase, pgvector, raw SQL, bcrypt, JWT

1. Abstract

MoodMix is a mood-aware playlist generator that converts a user's natural-language mood description into a personalized playlist. Instead of matching only exact keywords, the application creates a 384-dimensional embedding for the description and uses PostgreSQL's pgvector extension to retrieve songs by cosine similarity.

The project combines a normalized relational database with vector search. PostgreSQL stores users, roles, moods, songs, playlists, playlist membership, mood tags, statistics, and generation history. Next.js provides the web interface and API layer. Authentication, role-based access, transactions, triggers, views, full-text search, joins, subqueries, recursive CTEs, and vector indexes demonstrate the DBMS concepts required by the course.

2. Problem Statement

Traditional playlist systems often depend on manually selected genres, artists, or keywords. These choices do not always represent how a user feels. A user may want "calm music for a rainy evening" or "high-energy music for a workout" without knowing which genres or songs to select.

MoodMix addresses this problem by accepting a natural-language mood description, matching it semantically with a vector representation, and generating a ranked playlist from the song catalogue.

This is not only a CRUD application. It demonstrates a complete database-backed workflow with similarity search, relational modelling, data integrity, analytics triggers, authorization, and

transaction-safe playlist generation.

3. Objectives

1. Design a normalized relational schema for a mood-based music system.
2. Store and retrieve semantic embeddings using PostgreSQL and pgvector.
3. Generate playlists from natural-language mood descriptions.
4. Rank songs using cosine similarity and store their relevance scores.
5. Demonstrate DDL, DML, joins, aggregates, subqueries, recursive CTEs, functions, triggers, views, indexes, full-text search, and transactions.
6. Provide a functional Next.js interface connected to Supabase PostgreSQL.
7. Implement authentication with hashed passwords and two roles: USER and ADMIN.
8. Maintain playlist history and daily mood statistics.
9. Provide an auditable implementation that can be demonstrated through SQL and UI.

4. Scope

Included

- User registration and login
- JWT-based session cookie
- User and administrator roles
- Natural-language mood input
- Offline-friendly deterministic embedding generation
- pgvector cosine similarity search
- Playlist generation and storage
- Playlist song ordering and relevance scores

- Saving and deleting playlists
- Admin song API operations
- Daily generation and save statistics
- Full-text song search SQL
- Database-level keys, foreign keys, checks, indexes, and RLS policies

Not currently complete in the UI

- A visual admin song CRUD table/form
- A visual mood analytics dashboard
- A complete search page connected to the search API
- Automatic mapping of every free-text mood to a predefined `moods` row
- A separate final project report with screenshots and executed test evidence

The underlying database and APIs contain much of this supporting functionality, but the items above should be identified honestly during the demonstration.

5. Technology Stack and Justification

Layer	Technology	Reason
Frontend	Next.js 14 App Router	Page routing, server rendering, and client forms
Backend	Next.js Route Handlers	Keeps API and UI in one deployable application
Language	TypeScript	Type safety for pages, API responses, and session objects
Relational database	PostgreSQL through Supabase	Strong constraints, joins, transactions, functions, triggers, and RLS
Vector database capability	PostgreSQL `pgvector`	Stores embeddings beside relational song data and supports similarity search

Layer	Technology	Reason
Database client	pg	Raw SQL remains visible and gradeable
Password security	bcryptjs	Passwords are stored as hashes rather than plaintext
Authentication	jose signed JWT	Stateless, HTTP-only session cookie
Styling	CSS in app/globals.css	Lightweight project-specific interface styling

PostgreSQL with pgvector is appropriate because the project needs both relational integrity and semantic retrieval. Songs, users, and playlists need foreign keys and transactions, while mood descriptions and song representations need vector search.

6. System Architecture

```
Browser
   |
   v
Next.js pages in app/
   |
   v
Next.js API route handlers in app/api/
   |
   v
pg Pool in lib/db/pool.ts
   |
   v
Supabase PostgreSQL
   |-- relational tables
   |-- pgvector embeddings and IVFFLAT indexes
   |-- functions, triggers, views, RLS
```

Important implementation locations

- Application shell and authentication-aware navigation: app/layout.tsx
- Home mood prompt: app/page.tsx
- Login and registration: app/login/page.tsx, app/register/page.tsx
- Playlist dashboard: app/dashboard/page.tsx
- Playlist details: app/playlist/[id]/page.tsx
- Database connection: lib/db/pool.ts
- Session creation and verification: lib/auth/session.ts
- Password hashing: lib/auth/hash.ts
- Embedding generation: lib/embeddings/embed.ts

7. End-to-End Working Flow

7.1 Registration

1. The user submits username, email, and password in the registration page.
2. POST /api/auth/register validates required fields and password length.
3. bcryptjs hashes the password.
4. PostgreSQL inserts the user into users.
5. Unique constraints prevent duplicate usernames or emails.

Implementation: app/register/page.tsx and app/api/auth/register/route.ts.

7.2 Login

1. The user submits email and password.
2. POST /api/auth/login finds the user using a case-insensitive email lookup.
3. The submitted password is compared with the bcrypt hash.
4. A signed JWT containing user ID, username, and role is created.
5. The JWT is stored in an HTTP-only session cookie.

Implementation: app/login/page.tsx, app/api/auth/login/route.ts, and lib/auth/session.ts.

7.3 Playlist generation

1. The user enters text such as calm music for a rainy evening.
2. The home page sends the text to POST /api/playlists.
3. The API verifies the session.
4. embedText() converts the text into a 384-dimensional embedding.
5. get_similar_songs() ranks songs using cosine distance.
6. The API starts a database transaction.
7. A playlist is inserted into playlists.
8. Ranked songs are inserted into playlist_songs with positions and scores.
9. A generation event is inserted into playlist_history.
10. The transaction commits and the browser opens the playlist page.

Implementation: app/page.tsx, app/api/playlists/route.ts, and lib/embeddings/embed.ts.

7.4 Relevance score

The similarity returned by the database is:

```
1 - (song_embedding <=> query_embedding)
```

The <=> operator returns cosine distance. Therefore, a higher relevance score means the song is more semantically similar to the requested mood. Results are ordered with ORDER BY similarity DESC.

7.5 Save and delete

The playlist detail page allows the user to save or delete their own playlist. The API checks the session user ID and uses an ownership condition in the SQL statement. Saving changes is_saved; deleting removes the playlist and related playlist songs through cascading foreign keys.

Implementation: app/playlist/[id]/page.tsx and app/api/playlists/[id]/route.ts.

7.6 Logout

The header submits POST /api/auth/logout. The route redirects to / and clears the HTTP-only session cookie using the same cookie path (/) used during login.

Implementation: app/layout.tsx and app/api/auth/logout/route.ts.

8. Database Design

The complete schema is implemented in db/01_schema.sql.

8.1 Tables

Table	Purpose
users	Accounts, password hashes, roles, and timestamps
moods	Predefined moods, descriptions, and 384-dimensional embeddings
songs	Song catalogue, metadata, optional embeddings, and search vector
playlists	User-owned generated playlists and save state
playlist_songs	Many-to-many playlist/song relationship with position and score
song_moods	Many-to-many song/mood relationship with relevance
mood_stats	Daily generated and saved counters for each mood
playlist_history	Audit history for generation events

8.2 Keys and relationships

- Every entity uses a UUID primary key.
- playlists.user_id references users.user_id.
- playlists.mood_id references moods.mood_id.
- playlist_songs has a composite primary key: (playlist_id, song_id).

- song_moods has a composite primary key: (song_id, mood_id).
- mood_stats.mood_id references moods.mood_id.
- playlist_history records the user, optional playlist, and optional mood.

8.3 Integrity constraints

- NOT NULL protects required data.
- Unique constraints protect usernames, emails, mood names, and daily mood rows.
- CHECK (duration_seconds > 0) prevents invalid song durations.
- Relevance scores are constrained between 0.0 and 1.0.
- ON DELETE CASCADE removes dependent records where ownership requires it.
- ON DELETE SET NULL preserves history when an optional parent is deleted.

8.4 ER diagram and relational schema

- ER diagram: docs/er-diagram.png
- Relational schema: docs/relational-schema.md

9. SQL Implementation and Demonstration

9.1 DDL

db/01_schema.sql demonstrates:

- CREATE EXTENSION for pgcrypto and vector
- CREATE TYPE user_role
- CREATE TABLE
- Primary and foreign keys
- UNIQUE, NOT NULL, and CHECK constraints
- CREATE INDEX

- `ALTER TABLE` for the generated full-text search column

The setup is idempotent for most objects through `IF NOT EXISTS` and guarded type creation.

9.2 DML

db/02_seed_data.sql inserts demo moods, songs, mood relationships, and users.

db/03_dml_examples.sql demonstrates:

- `INSERT ... RETURNING`
- `UPDATE`
- `DELETE`
- `SELECT`
- `JOIN`
- `GROUP BY`
- `HAVING`
- `ORDER BY`
- aggregate functions
- correlated `NOT EXISTS` subquery
- nested subquery
- recursive CTE

9.3 Joins and aggregates

Example from the project:

```
SELECT p.name, s.title, s.artist, m.mood_name
FROM playlists p
JOIN playlist_songs ps ON ps.playlist_id = p.playlist_id
JOIN songs s ON s.song_id = ps.song_id
LEFT JOIN song_moods sm ON sm.song_id = s.song_id
LEFT JOIN moods m ON m.mood_id = sm.mood_id;
```

This combines playlist, membership, song, and mood data.

9.4 Correlated and nested queries

Users with no saved playlist:

```
SELECT u.username
FROM users u
WHERE NOT EXISTS (
  SELECT 1
  FROM playlists p
  WHERE p.user_id = u.user_id AND p.is_saved
);
```

The inner query depends on the current row of the outer `users` query.

9.5 Recursive CTE

The project creates a seven-day date range and combines it with moods. This is useful for reporting days where no activity exists yet:

```
WITH RECURSIVE days(d) AS (
  SELECT CURRENT_DATE - 6
  UNION ALL
  SELECT d + 1 FROM days WHERE d < CURRENT_DATE
)
SELECT d, m.mood_name
FROM days CROSS JOIN moods m
ORDER BY d;
```

10. Advanced Database Concepts

The implementation is in db/04_advanced.sql.

10.1 Transactions

Playlist generation uses `BEGIN`, inserts the playlist, inserts its songs, inserts history, and then uses `COMMIT`. If any operation fails, the API executes `ROLLBACK`, preventing a partially

generated playlist.

10.2 Stored functions

record_generation() increments the daily generated count.
record_save() increments the daily saved count only when a playlist changes from unsaved to saved.
get_similar_songs() returns songs ordered by vector similarity.

10.3 Triggers

- playlist_generation_stats runs after a playlist_history insert.
- playlist_save_stats runs after playlists.is_saved is updated.

The triggers maintain mood_stats transactionally.

10.4 View

public_playlists_view joins users, playlists, moods, and playlist songs and returns a safe public projection without exposing password hashes.

10.5 Indexing

The schema creates:

- IVFFLAT index on song embeddings
- IVFFLAT index on mood embeddings
- Playlist owner index
- Playlist-song song index
- GIN index on the generated full-text search vector

10.6 Full-text search

The songs.search_vector generated column combines title, artist, and lyrics. The GIN index supports queries using websearch_to_tsquery.

11. Vector Database and Semantic Search

Data converted to embeddings

- Natural-language mood prompts are embedded at request time.
- Seeded moods and songs contain 384-dimensional vectors.

Embedding approach

The project uses a deterministic offline-friendly embedding implementation in lib/embeddings/embed.ts. The schema and SQL functions use the 384-dimensional format associated with the MiniLM strategy. This keeps the demonstration reproducible without requiring an external model API.

Storage

Embeddings are stored in PostgreSQL `vector(384)` columns:

- `moods.embedding`
- `songs.embedding`

Similarity metric

Cosine distance is used through `vector_cosine_ops` and the `<=>` operator.

Retrieval

`get_similar_songs(query_embedding, result_limit)` filters out songs without an embedding, orders by cosine distance, and returns the top results.

The SQL demonstration is in db/05_vector_search.sql.

12. Authentication, Authorization, and Security

Authentication

- Passwords are hashed with bcrypt.
- Password hashes are never returned by API responses.

- JWTs are signed with `AUTH_SECRET`.
- The session cookie is HTTP-only.
- Production cookies use the `secure` flag.
- Sessions expire after seven days.

Authorization

- Normal users can access only their own playlists.
- Admin users can access `/admin`.
- Middleware protects `/admin` routes.
- Song mutation routes require the `ADMIN` role.
- Playlist API routes check the authenticated user ID.

Database role model

The `user_role` enum restricts roles to `USER` and `ADMIN`. RLS policies are defined in db/06_rls_policies.sql.

Important implementation note

The API currently performs ownership and role checks in application code. The RLS policies use PostgreSQL session settings (`app.user_id` and `app.role`), but the connection pool does not currently set those values for every request. For a production-strength RLS demonstration, the API should set the request identity inside a transaction before querying protected tables.

Configuration security

Database credentials and `AUTH_SECRET` belong in `.env.local`; they are not hard-coded in source files. `.env.local` is ignored by Git.

13. Application Pages and API Routes

Pages

Page	Purpose
/	Natural-language mood prompt and playlist generation
/login	User login
/register	Account registration
/dashboard	User's playlists
/playlist/[id]	Playlist details, save, and delete actions
/admin	Admin console
/admin/songs	Admin catalogue information page

API routes

Route	Purpose
POST /api/auth/register	Create account
POST /api/auth/login	Authenticate and set session
POST /api/auth/logout	Clear session
GET /api/playlists	List current user's playlists
POST /api/playlists	Generate a playlist
GET/PATCH/DELETE /api/playlists/[id]	Read, save/update, or delete an owned playlist
GET/POST/PATCH/DELETE /api/songs	Read or administer songs
GET /api/search?q=...	Vector song search
GET /api/moods	Read moods
GET /api/health	Check database connectivity

14. Setup and Demo Procedure

14.1 Configure Supabase

1. Create a Supabase project.

2. Enable PostgreSQL and install the vector extension through the SQL setup.

3. Copy the Session Pooler URI.

4. Add the values to .env.local:

```
DATABASE_URL=your-supabase-session-pooler-uri
AUTH_SECRET=use-a-long-random-secret
```

14.2 Install and initialize

```
npm install
.\scripts\db-setup.ps1
```

Alternatively, execute the files under db/ in this order:

1. 00_extensions_check.sql
2. 01_schema.sql
3. 02_seed_data.sql
4. 04_advanced.sql
5. 06_rls_policies.sql

Run 03_dml_examples.sql and 05_vector_search.sql manually as demonstrations.

14.3 Start the application

```
npm run dev
```

Open http://localhost:3000.

14.4 Suggested live demonstration

1. Open /api/health and show { "ok": true }.
2. Register a new account.
3. Log in and show the session-aware header.
4. Enter calm music for a rainy evening.
5. Show the generated playlist and ranked relevance scores.
6. Save the playlist.
7. Open the dashboard and show the saved playlist.
8. Delete the playlist and show ownership-protected deletion.

9. Log out and show that the header returns to `Sign in`.

10. Log in as an admin and open `/admin`.

11. In Supabase SQL Editor, demonstrate a join, vector query, trigger function, view, and mood statistics query.

14.5 Useful SQL during the demo

```
SELECT * FROM public_playlists_view;

SELECT m.mood_name, ms.stat_date, ms.generated_count, ms.saved_count
FROM mood_stats ms
JOIN moods m ON m.mood_id = ms.mood_id
ORDER BY ms.stat_date DESC;

SELECT song_id, title, artist,
       1 - (embedding <=> '[...]') AS similarity
FROM songs
WHERE embedding IS NOT NULL
ORDER BY embedding <=> '[...]'
LIMIT 5;
```

Replace `[...]` with a valid 384-dimensional vector when running the last query.

15. Requirement Coverage Matrix

Requirement from the course document	**Status**	**Implementation**
Meaningful real-world problem	Complete	Mood-aware semantic playlist generation
Two-member team	Complete	`docs/contributions.md`
SQL database	Complete	Supabase PostgreSQL
Vector database	Complete	pgvector in `db/01_schema.sql`
ER diagram	Complete	`docs/er-diagram.png`
Relational schema	Complete	`docs/relational-schema.md`

Requirement from the course document	**Status**	**Implementation**
Entities and attributes	Complete	db/01_schema.sql
Primary and foreign keys	Complete	All core tables
Constraints	Complete	Unique, not-null, check, and foreign-key constraints
CREATE	Complete	Schema and extension setup
ALTER	Complete	Generated search_vector column
DROP	Demonstrated	Trigger recreation uses DROP TRIGGER IF EXISTS
Indexes	Complete	IVFFLAT, GIN, and relational indexes
Views	Complete	public_playlists_view
INSERT/UPDATE/DELETE/SELECT	Complete	db/02_seed_data.sql, db/03_dml_examples.sql, API routes
Joins and aggregates	Complete	db/03_dml_examples.sql
GROUP BY and HAVING	Complete	db/03_dml_examples.sql
Subqueries	Complete	Correlated and nested examples
Transactions	Complete	Playlist generation API
Stored functions	Complete	db/04_advanced.sql
Triggers	Complete	Generation and save statistics
Full-text search	Complete in SQL	search_vector, GIN index, SQL example
Vector similarity search	Complete	API and db/05_vector_search.sql
Next.js interface	Complete	app/
Backend/API integration	Complete	app/api/ and lib/db/
Environment variables	Complete	.env.local, .env.example, README
Authentication	Complete	bcrypt and JWT
Minimum two roles	Complete	USER and ADMIN enum and route checks

Requirement from the course document	Status	Implementation
Database-level validation	Mostly complete	Constraints are present; more API validation can be added
Password protection	Complete	bcrypt hashing
Problem/objectives/scope	Documented here	This report
Data dictionary	Partial	Schema files and relational schema exist; expand for final submission
System architecture	Documented here	This report
Application screenshots	Missing	Capture screenshots during the final demo
Testing results	Partial	Production build passes; add scenario test evidence
Conclusion/future enhancements	Documented below	This report
Git/GitHub	Complete	Repository history and contribution document
Individual contribution	Complete	docs/contributions.md

16. Testing and Validation

Verified

- Production build completed successfully with `npm.cmd run build`.
- Type checking completed as part of the Next.js build.
- Next.js routes were generated successfully.
- Logout cookie path was corrected so the session cookie is cleared using `/`.

Recommended demo test cases

Test	Expected result
Register valid user	Account is created

Test	Expected result
Register duplicate email	Conflict error is returned
Login with wrong password	401 Invalid credentials
Generate without login	401 Sign in before generating
Generate with valid mood	Playlist and songs are created
Save own playlist	is_saved becomes true
Delete own playlist	Playlist is removed
Access another user's playlist	Request is rejected or returns unavailable
Access admin as normal user	Redirect to login / denied
Logout	Session cookie is cleared
Database unavailable	Health/API returns a clear error
Invalid song duration	Database check constraint rejects it
Duplicate mood name	Unique constraint rejects it

17. Known Gaps and Recommended Improvements

1. **Mood statistics coverage:** the UI currently sends free-text mood input and does not always provide a predefined mood_id. As a result, generation statistics may not update for free-text generations. Resolve this by selecting the closest mood before inserting the playlist and history row.

2. **Admin UI:** implement a table and forms for song create, update, and delete instead of exposing only API instructions.

3. **Analytics UI:** add an admin dashboard showing mood_stats.

4. **Search UI:** connect the existing /api/search endpoint to a page with a search input and ranked results.

5. **RLS request context:** set app.user_id and app.role on the database connection inside each protected transaction.

6. **Documentation evidence:** add screenshots, executed SQL output, and a test-case table with actual dates/results before final submission.

7. **Production embeddings:** replace the deterministic fallback with a documented production embedding model if external model hosting is available.

These are improvement opportunities rather than reasons the core prototype cannot be demonstrated.

18. Conclusion

MoodMix satisfies the central DBMS objective: it connects a meaningful application problem to a designed, implemented, and queryable database. The project goes beyond basic CRUD through semantic vector search, cosine similarity ranking, transactions, triggers, functions, views, full-text search, recursive queries, and role-aware access.

The strongest demonstration sequence is to show the natural-language request, the vector-ranked songs, the relational rows created by one request, and the SQL features that maintain and analyze those rows. The remaining work is primarily presentation/UI completeness: screenshots, test evidence, a statistics dashboard, and richer admin/search interfaces.

19. Viva Preparation Questions

Why PostgreSQL plus pgvector?

PostgreSQL provides relational integrity and advanced SQL, while pgvector adds semantic similarity search without maintaining a separate database for this scale.

Why cosine similarity?

Cosine similarity compares the direction of embedding vectors and is commonly used for semantic text matching. A smaller cosine distance means a closer semantic match.

Why is the relevance score higher for better matches?

The project stores `1 - cosine_distance`, so a closer vector produces a larger score. Results are sorted in descending order.

Why use a junction table?

One playlist contains many songs and one song can occur in many playlists. The `playlist_songs` table represents this many-to-many relationship and stores relationship-specific

attributes such as position and relevance score.

Why use a transaction during generation?

Playlist, playlist-song, and history rows must either all be created or none should be created. A transaction prevents partial data when one insert fails.

What do the triggers do?

They maintain daily generated and saved mood counters automatically whenever generation history or a playlist save event is inserted.

How are passwords protected?

Passwords are hashed with bcrypt before insertion. The application never stores or returns plaintext passwords.

How is an admin different from a user?

The database role enum stores `USER` or `ADMIN`. Middleware and API route checks restrict admin routes and song mutations to administrators.

What is the difference between vector and full-text search?

Full-text search matches linguistic terms and indexed words. Vector search compares semantic meaning, so related phrases can match even when they use different words.